

Project Documentation

AI BlogNest API

INTRODUCTION

- **Project Title:** AI BlogNest API
- **Team ID:** SWTID-2026-6909
- **Team Leader:** GUNAVATHI S & gunapoppz@gmail.com
- **Team member:** SIVASANKARI T & sivasankarithennarasu23@gmail.com
SIVASAKTHI T & sivasakthithennarasu76@email.com
LOGITHA P & logiprasad30@gmail.com

AI BLOGNEST – GENERATE HIGH-QUALITY BLOG CONTENT USING AI

DESCRIPTION

The BlogNest backend is a secure, intelligent REST API platform built to make blog creation, content management, and AI-assisted writing easier. It is developed with Node.js and Express.js and stores its data in MongoDB, using the Mongoose ODM for data modeling, validation, and smooth database interaction.

The platform is integrated with Google Gemini AI, which powers features such as AI-based blog content generation, an AI FAQ assistant, AI WeatherWise insights, and AI FitTrack recommendations. These features let users produce high-quality, context-aware content with very little effort.

Security is central to the design. Stateless authentication is handled with JWT (JSON Web Tokens), authorization uses role-based access control (Admin, Editor, Author, Reader), and passwords are protected with bcrypt.js hashing. The backend also supports complete blog management, categories and tags, comment management, media metadata handling, advanced search, and centralized error handling. Request validation, input sanitization, rate limiting, and structured logging further keep the platform reliable, scalable, and protected against common web vulnerabilities.

Thanks to its modular and scalable structure, BlogNest works as a modern AI-powered Content Management System (CMS). It combines traditional blogging features with Generative AI to give users a faster, smarter, and more personalized way to create content.

SCENARIO-BASED CASE STUDY

BACKGROUND

Priya is a content creator who runs several blogs. She finds it hard to organize her posts, schedule publications, manage categories and tags, and keep track of how readers engage with her content.

PROBLEM

- Managing many drafts, categories, and tags is difficult.
- There is little visibility into post performance and engagement analytics.
- Content approval and comment moderation take too much time.
- There is no streamlined way to handle media uploads and their metadata.
- There are no collaborative permission levels for editors, authors, and administrators.

SOLUTION

The BlogNest backend provides a centralized and secure API layer that separates application business logic from frontend delivery. It introduces an internal permission matrix that controls

dedicated services for content scheduling, category taxonomy, media path registration, and fine-grained user roles.

USAGE

- **Author:** Creates blog posts, uploads media paths, organizes content drafts, and reviews the performance data of their own posts.
- **Editor/Admin:** Access protected endpoints to review the content queue, change publishing status, moderate visitor comments, and upgrade user privileges.
- **Reader:** Queries the public list routes to read posts and uses the comment creation endpoints to respond.

OUTCOME

By moving workflow control into a robust backend, delays in content delivery are kept to a minimum. Schema validation preserves the structural integrity of the data, security filters ensure that only authorized roles can carry out administrative actions, and persistent tracking supplies raw analytics for accurately measuring post performance.

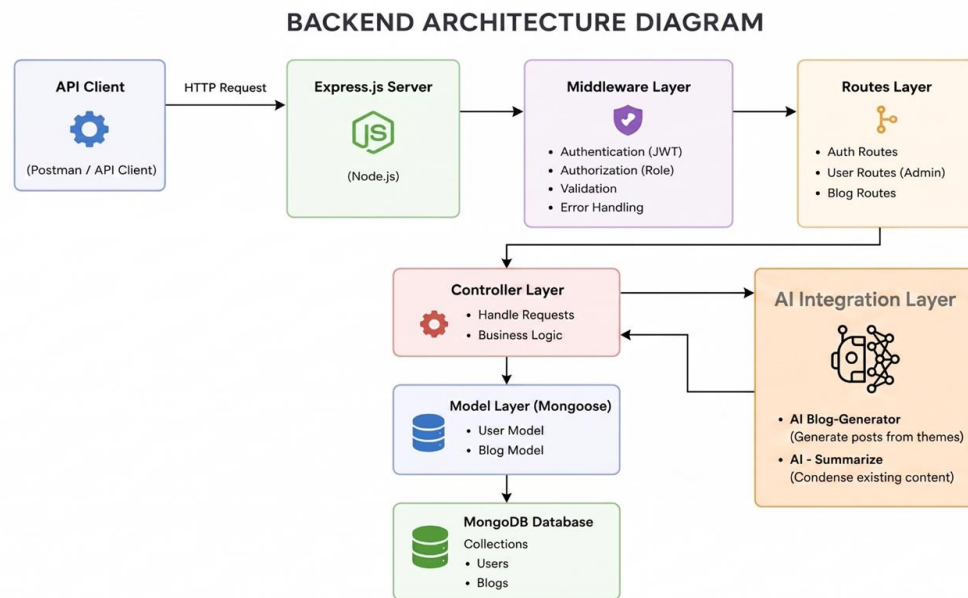
1. SOFTWARE REQUIREMENTS

- **Operating System:** Windows 10/11, macOS, or Linux (cross-platform support).
- **Node.js:** Version 16 or above; the runtime that executes the server logic and routing.
- **npm:** Version 8 or above; the package manager used to manage project dependencies.
- **Express.js:** A lightweight web framework used to build the backend RESTful endpoints.
- **MongoDB:** The NoSQL database that stores users, posts, categories, comments, and analytics data.
- **Postman:** An API testing tool used to verify schema validation and responses from the role-protected routes.
- **Code Editor:** Visual Studio Code or a similar IDE.

2. HARDWARE REQUIREMENTS

- **Processor:** Intel Core i5 (8th generation or above), AMD Ryzen 5, or equivalent.
- **RAM:** Minimum 8 GB (16 GB recommended for running MongoDB, processing layers, and application instances together).
- **Storage:** At least 1 GB of free disk space.

EPIC 1: PROJECT ARCHITECTURE



The BlogNest application exposes structured endpoints through a layered server architecture. Every incoming request passes through a series of filters before it reaches the data services.

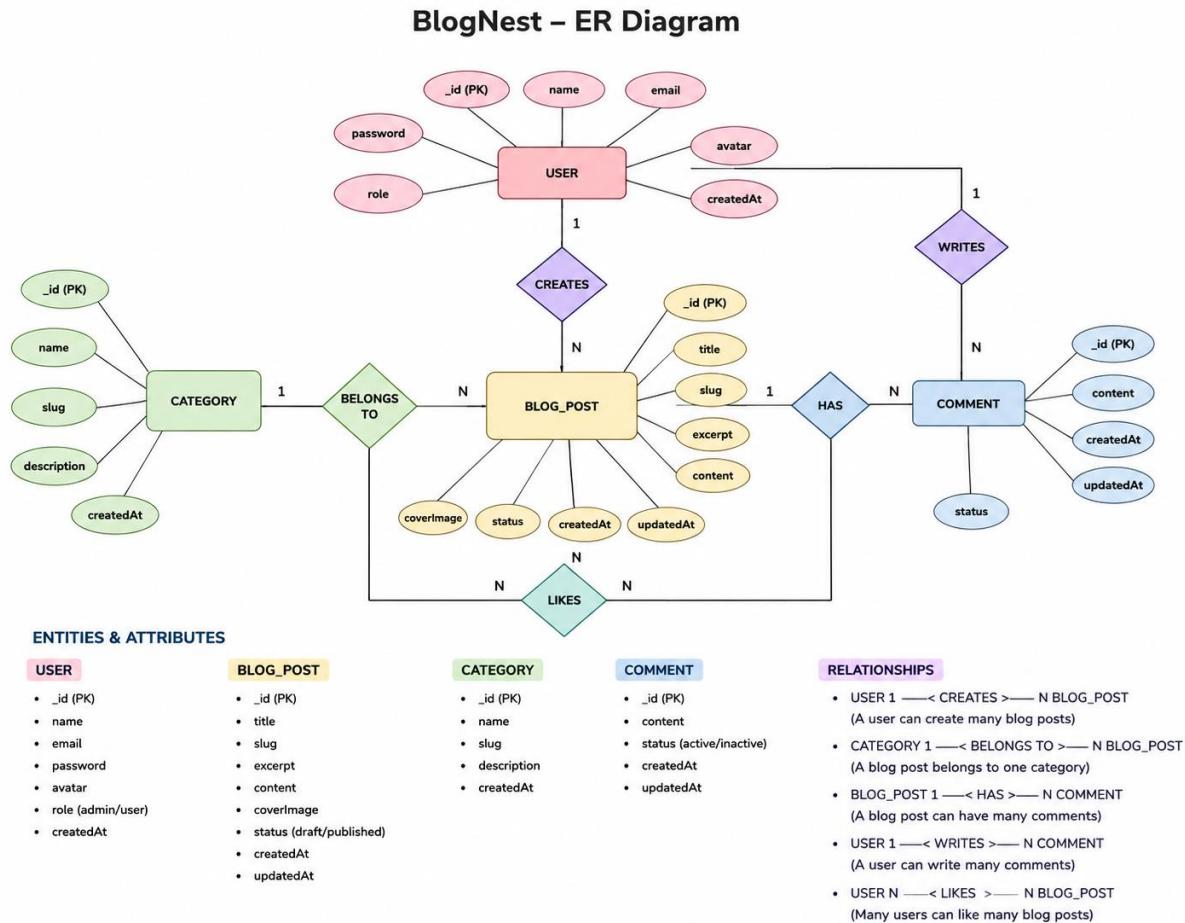
COMPONENTS DEFINITION

- **Interface:** API tools such as Postman send requests to specific routes and receive JSON responses.
- **Express Server Gateway:** Listens on the network port, maps HTTP methods (GET, POST, PUT, DELETE) to routes, parses JSON payloads, and handles CORS headers.
- **Authentication and Role Matrix Middleware:** Reads tokens from request headers, verifies the user's identity, and checks whether the user's role meets the authorization level required for protected resources.
- **Granular Services (Logic Modules):** Apply separate domain rules for the blog lifecycle (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose converts application logic into efficient native MongoDB queries.
- **AI Flow (Service Integration):** Provides intelligent content automation, including automated blog generation and real-time content summarization. This layer connects to external AI models through the dedicated endpoints `/api/ai/generate-blog` and `/api/ai/summarize`, allowing the controller layer to hand off complex text-processing work and improve content creation workflows.

- **BlogGenerator with AI:** A specialized service that automates the creation of blog content and works together with the AI Flow service to speed up the drafting process.

ENTITY-RELATIONSHIP (ER) DIAGRAM DESCRIPTION

The underlying data structure consists of several collections that are linked through logical application references:



1. USER ENTITY

- _id (Primary Key – ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. POST ENTITY

- _id (Primary Key – ObjectId)

- userID (Foreign Key referencing the User collection)
- name (String, Required – represents the title)
- photo (Object/String – path or link to the image)
- message (String – main content body)
- likes (Number, Default: 0)
- timestamps (True)

KEY STRUCTURAL MAPPINGS

- **User to Post (One-to-Many):** One author can create many blog posts.
- **User to Comment (One-to-Many):** Readers and users can add many comments across different posts.
- **Post to Comment (One-to-Many):** A single blog post can hold many individual comments.

KEY FEATURES

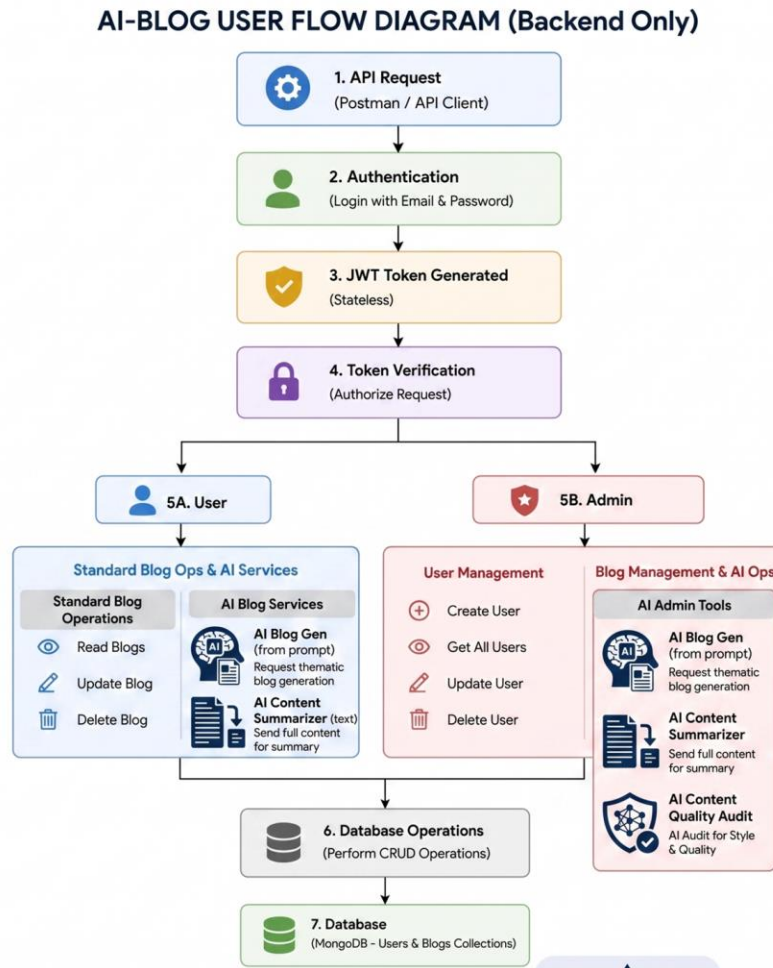
- **Role-Based Route Guarding:** Strict token processing prevents unauthorized roles from changing core settings.
- **CRUD Blog Execution Engine:** Complete management of content definitions, status states, and media asset paths.
- **Categorization System:** Built-in tag indexing supports filtering and searching, which improves discoverability.
- **Comment Moderation Logic:** Protected endpoints for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Sanitization of request parameters protects against XSS and NoSQL injection attacks.
- **AI-Driven Blog Generation Pipeline:** Connects to external Large Language Model (LLM) services and uses context-aware prompt templates to create full-length, structured blog posts from high-level topics.
- **Intelligent Content Summarization Engine:** Uses semantic compression to process long articles and automatically produce short abstracts and metadata summaries.

ROLES AND RESPONSIBILITIES

- **Admin:** Has full control over the environment. Can manage user registrations, change role assignments, remove malicious comments, and inspect complete logs.
- **Editor:** Handles editorial workflows. Reviews pending draft posts, publishes approved records immediately, and takes care of category cleanup.

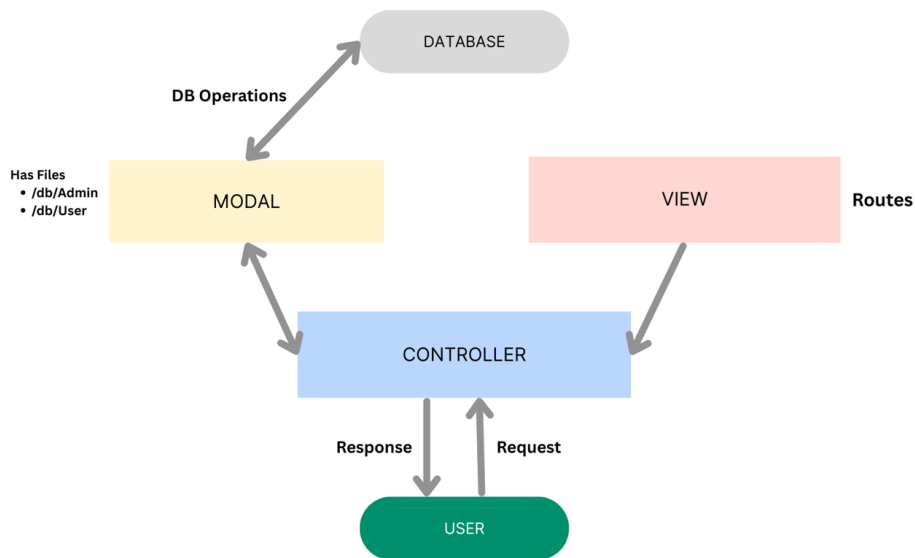
- **Author:** Has restricted access. Can create posts only under their own account.
- **Reader:** The consumer profile. Reads published content and posts comments under an authenticated identity.

USER FLOW



MVC ARCHITECTURE PATTERN

The system is organized around the Model-View-Controller (MVC) design pattern so that its components remain fully decoupled:



MODEL LAYER (DATA BLUEPRINT)

This layer defines the data structure using strictly enforced Mongoose schemas that map fields to documents in MongoDB. It sets default values, indexing rules, and transformation parameters before data is validated and saved.

CONTROLLER LAYER (EXECUTION MATRIX)

The controller is the central coordinator of the system. It receives requests from the routes, checks the structure of each request, passes the processing work to the models, and returns the results as standard JSON responses.

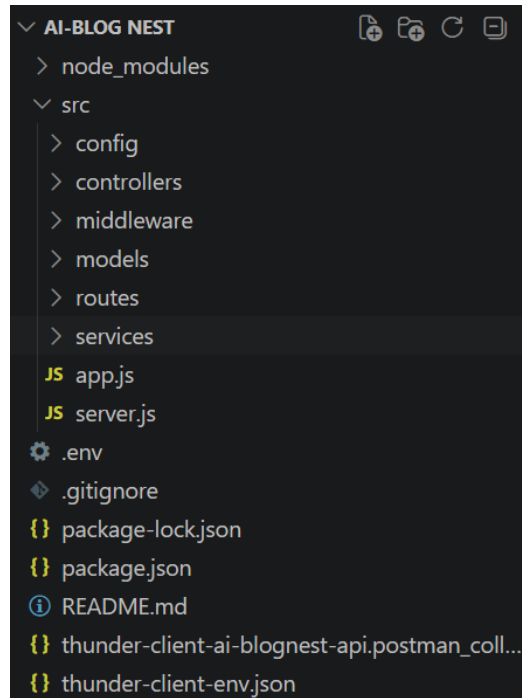
VIEW LAYER (ROUTING NETWORK INTERFACE)

Because BlogNest is a purely headless backend, it has no user-interface templates. Instead, the view layer takes the form of an API routing layer that connects requests on specific paths to the matching controller handlers.

EPIC 2: PROJECT SETUP AND CONFIGURATION

CREATING THE PROJECT FOLDER

1. Create a new folder named after your project.
2. Inside that folder, create a new sub-folder.
3. Name that sub-folder "Server".
4. Open the folder in VS Code.



SERVER SETUP (NPM INIT)

- Open the Server folder in the VS Code terminal and run: `npm init -y`
- Create the file: `server.js`
- Create the folders: `models`, `controllers`, and `routes`

EPIC 3: BACKEND DEVELOPMENT

SERVER.JS

```
JS server.js  X
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10 |   console.log(`Server running on port ${PORT}`);
11 | });
12
```

APP.JS

```
JS app.js ×
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
26
```

EPIC 4: DATABASE CONFIGURATION

DB CONNECTION

```
Explorer (Ctrl+Shift+E)
src > config > JS dbjs > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
```

1. CONFIGURE MONGODB

- Install Mongoose. In your Node.js project directory, run: `npm install mongoose`
- Create the database connection.
- Create the schemas and models.

CREATE THE DATABASE CONNECTION

Before performing any backend operation, make sure the database is connected. The connection code is similar to the example shown above.

MONGOOSE SCHEMA DEFINITIONS

1. IDENTITY RESOURCE DOCUMENT SCHEMA (SCHEMAS/USERMODEL.JS)

This schema enforces field-level validation whenever a new record is created:

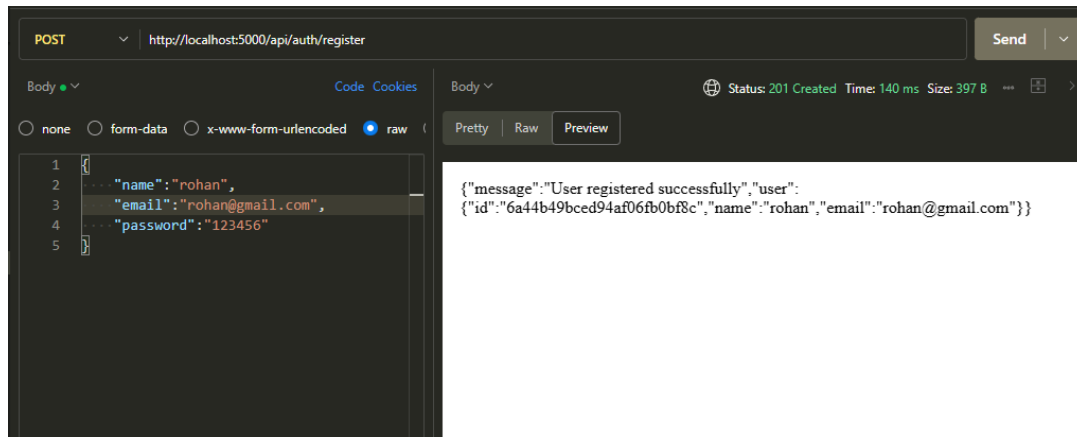
```
JS User.js  ×
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

EPIC 5: PROJECT EXECUTIONS

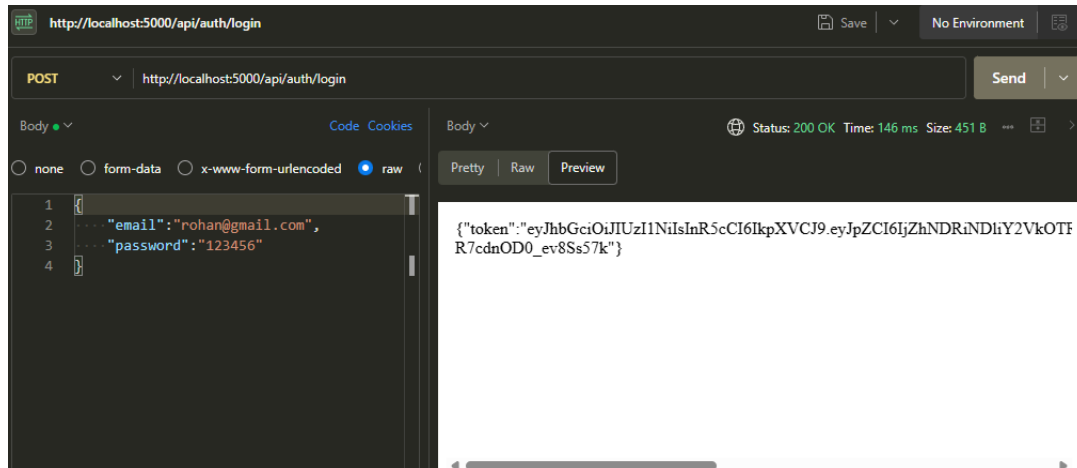
- Install Postman or ThunderClient (a VS Code extension).
- Start the backend server with the command: `npm start`

API TESTING

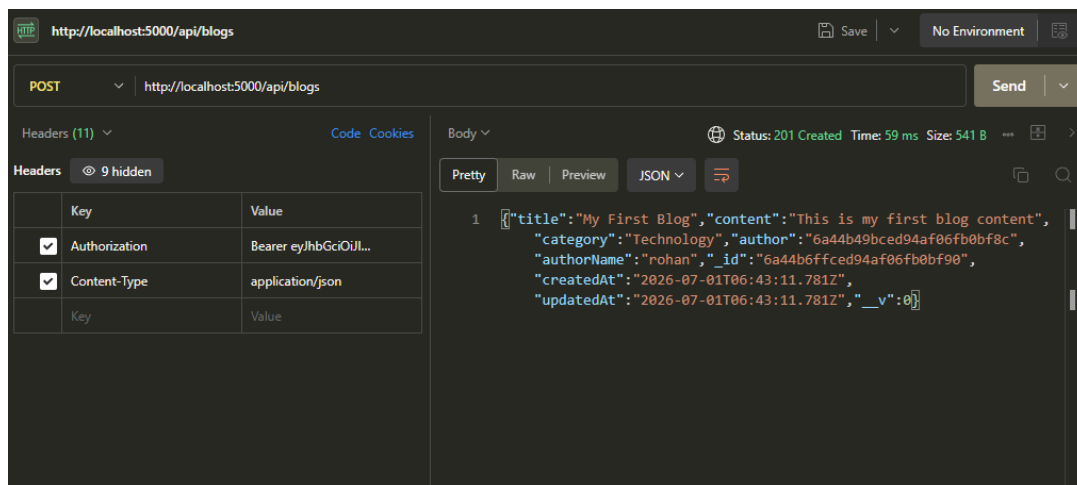
USER REGISTRATION



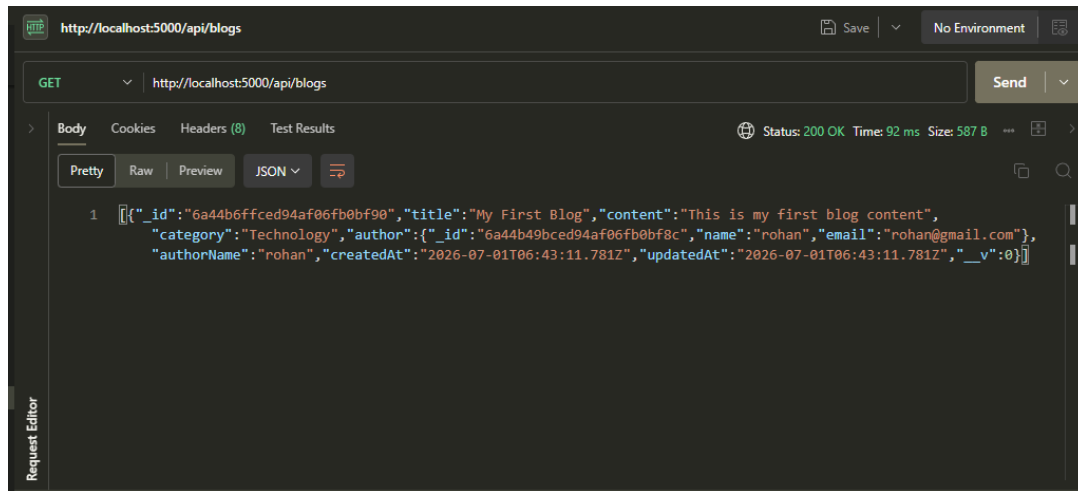
USER LOGIN



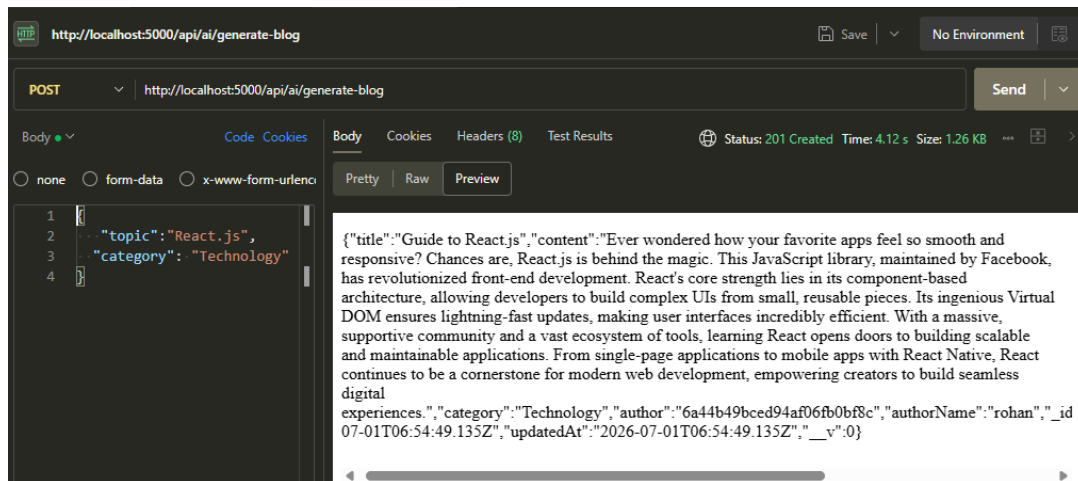
CREATE BLOGS



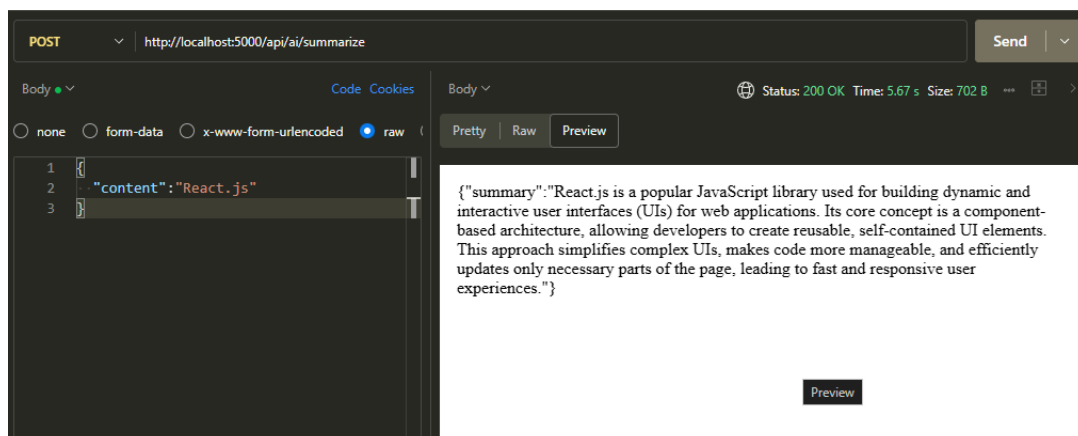
FETCH BLOGS



GENERATE AI BLOGS



SUMMARIZE BLOGS



PROJECT LINKS

DemoVideo: https://drive.google.com/file/d/1dgs0Hc68QgAInRHvYWUTxC_Othkeh2r-/view?usp=drivesdk